# AI Puzzle Escape: Generating Uniquely Solvable Puzzles with Constraint Satisfaction and Backtracking, and Adapting Their Difficulty to the Player

**GAYATHRI MOHANRAAJ**
COMPUTER SCIENCE AND ENGINEERING
RAJALAKSHMI ENGINEERING COLLEGE
THANDALAM, INDIA
gayathrimohanraaj.2024.cse@rajalakshmi.edu.in

**Dr. Mrs. THEEPA KARTHISAI**
Assistant Professor
COMPUTER SCIENCE AND ENGINEERING
RAJALAKSHMI ENGINEERING COLLEGE
THANDALAM, INDIA

**DURKASRI SM**
COMPUTER SCIENCE AND ENGINEERING
RAJALAKSHMI ENGINEERING COLLEGE
THANDALAM, INDIA
durkasri.sm.2024.cse@rajalakshmi.edu.in

**A.Abstract**

Escape-room games usually ship hand-written puzzles whose answers are stored in the program, so every replay is identical and the player never sees how the computer reasons. This paper presents AI Puzzle Escape, a browser-based four-stage escape game in which every puzzle is generated and verified at run time by a Constraint Satisfaction Problem (CSP) solver that uses backtracking search. The engine picks a random secret solution, derives clues that are true for it, and adds them one at a time until a second backtracking pass finds exactly one remaining solution, so each puzzle is fair and no answer is hard-coded. The same constraint checker also validates the player's answers. Three puzzle families are modelled, a four-digit Number Lock, an Object Arrangement and a Logic Door puzzle, and an inventory chains them into one final code. An adaptive difficulty module watches attempts, solving time and hints, changes the number of extra clues for the next puzzle, and unlocks an extra hint when the player is stuck. A live AI Status panel and an explanation screen expose backtracks and constraint checks to the player. In 14,000 generated puzzles the engine produced a uniquely solvable puzzle every time, confirmed by an independent brute-force check. Mean generation time was 1.0 ms for the arrangement puzzle and 2.1 to 2.6 ms for the number lock, and the door generator needed 1.93 attempts on average. The whole system runs client-side in HTML, CSS and JavaScript.

***Keywords:*** *Constraint satisfaction, Backtracking, Procedural puzzle generation, Unique solution, Adaptive difficulty, Educational game, JavaScript*

## I. INTRODUCTION

Escape-room and puzzle games are a popular way to practise logical thinking, but most digital versions ship a fixed set of puzzles with the answers stored in the code. Once a player has seen them, the replay value is gone, and the "AI" in such a game is usually a script. In a Foundations of Artificial Intelligence course, on the other hand, Constraint Satisfaction Problems (CSPs) and backtracking are taught through textbook examples such as map colouring and Sudoku [1], and students rarely see them working inside a complete application.

This work builds AI Puzzle Escape to close that gap. The player is locked in a vault and must pass four stages: a Number Lock, an Object Arrangement, a set of Logic Doors and a Final Escape. The first three puzzles are not written by hand. A CSP engine creates each one at run time, proves that it has exactly one answer, and later checks the player's answer with the same rules. The items won in the first three stages (a lock code, the position of the key and the exit door) are combined into the final code.

Given a puzzle family described by variables, domains and a pool of constraint templates, the system must (a) generate consistent clues, (b) guarantee exactly one solution, (c) validate answers without a stored answer key, (d) adjust the number of clues to the player's performance, and (e) make the AI's reasoning visible. The main contributions are:

- A generate-and-verify method that grows a clue set until backtracking reports a single solution.
- One constraint checker, `checkConstraints()`, shared by the solver, the uniqueness test and the player's answer check.
- CSP models of three puzzle families, chained through an inventory into a final code.
- An adaptive difficulty module with struggle detection that unlocks an extra hint.
- A transparent AI Status panel and explanation screen that show backtracks and constraint checks.
- An evaluation of 14,000 generated puzzles, with uniqueness confirmed by independent brute force.

## II. LITERATURE SURVEY

Prior work relevant to this project falls into four areas: CSP solving, procedural puzzle generation, dynamic difficulty adjustment and the research gap that remains.

### *A. Constraint Satisfaction and Backtracking*

A CSP is defined by variables, a domain of values for each variable and constraints over them; a solution assigns every variable a value without breaking any constraint [1]. Backtracking is a depth-first search that assigns one variable at a time and undoes a choice when a constraint fails. It is improved by consistency techniques [2], and by forward checking with a fail-first variable ordering, which greatly reduces tree search effort [3]. Book-length treatments are given by Dechter [4], Tsang [5] and the Handbook of Constraint Programming [6]. A key property used here is that a rule can be tested as soon as all of its variables are assigned, which allows early pruning.

### *B. Procedural Puzzle Generation*

Togelius et al. surveyed search-based procedural content generation, in which candidate content is generated and then evaluated against requirements [7]. Smith and Mateas showed how answer set programming can describe the space of valid game content declaratively [8]. For puzzles, a central requirement is a unique solution. Yato and Seta studied the complexity of finding another solution to a puzzle, which is the question a uniqueness test must answer [9]. Our generator uses the simplest practical version of this idea: ask a complete solver for a second solution and stop adding clues when none exists.

XXX-X-XXXX-XXXX-X/XX/$XX.00 ©20XX IEEE

### C. Dynamic Difficulty Adjustment

Hunicke and Chapman proposed adjusting game difficulty from measured player performance so that the game stays challenging but not frustrating [10]. This follows the idea of flow, a state in which challenge matches skill [11], which was later adapted into heuristics for evaluating games [12]. Our module uses three cheap signals, attempts, time and hints, and changes the number of clues rather than the puzzle type.

### D. Research Gap

Many puzzle games either store fixed answers or generate random puzzles without checking how many solutions they have. Few small educational games show the CSP search to the learner. AI Puzzle Escape combines run-time generation, a proven unique solution, adaptive difficulty and a visible AI panel in a server-free web page. Table I compares the approaches.

**TABLE I. COMPARISON OF APPROACHES**

| Approach | Strength | Limitation | Use here |
|---|---|---|---|
| Hard-coded puzzles | Easy to author | Fixed; answer stored in code | Replaced |
| Random puzzle, no check | Fast | May have zero or many answers | Rejected |
| Search-based PCG / ASP [7], [8] | Expressive | Needs extra tooling | Idea reused |
| CSP + backtracking (this work) | Unique answer; explainable; client-side | Exponential worst case; small puzzles | Core engine |

## III. METHODOLOGY FOR PUZZLE GENERATION

### A. Overview

AI Puzzle Escape is a single-page web application with no back end. `index.html` holds the screens (start, game, explanation and victory), `style.css` the visual design, and `script.js` the AI engine, the generators, the adaptive module and the user interface. Fig. 1 shows the structure. All game state is kept in one JavaScript object, so the game needs no database and no install.

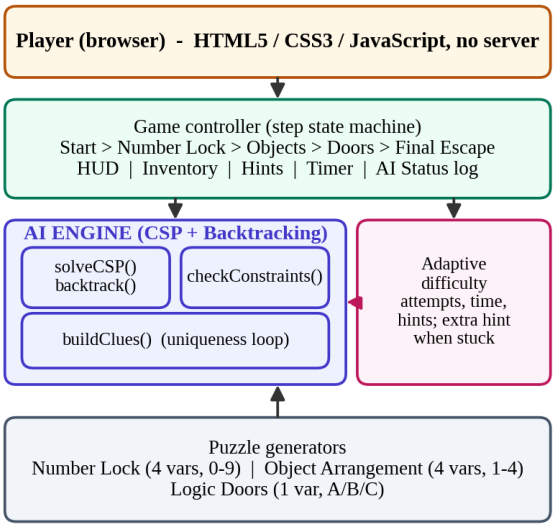



**Fig. 1. Architecture of AI Puzzle Escape**

**TABLE II. TECHNOLOGY STACK**

| Layer | Technology | Purpose |
|---|---|---|
| Structure | HTML5 | Screens and overlays |
| Style | CSS3 (grid, flexbox, animation) | HUD, responsive layout, reduced-motion support |
| Logic | JavaScript (ES6), no libraries | Engine, generators, UI |
| AI method | CSP with backtracking | Generate, verify and check |
| Runtime | Any modern browser | No server, no install |

### B. Game Flow

The game has four stages. In stage 1 the player enters a four-digit code from clues. In stage 2 the player places four objects (Book, Key, Coin, Card) in positions 1 to 4. In stage 3 the player chooses the exit among three doors whose signs make statements about the exit. Each solved stage adds an item to the inventory. In stage 4 the player types the lock code, the exit door letter and the position of the Key as one string such as `1234A1`. This forces the player to use all three earlier results.

### C. CSP Formulation

Each puzzle is a CSP. Table III lists the variables and domains. In the Number Lock the four variables are the digits and the domain is 0 to 9. Because the digits must differ, the search space is 5,040 candidates instead of 10,000. In the Object Arrangement the variables are the four objects and the values are positions 1 to 4, which gives 24 permutations. In the Logic Doors puzzle there is one variable, the exit door, with domain {A, B, C}.

**TABLE III. CSP MODEL OF EACH PUZZLE**

| Puzzle | Variables | Domain | Constraints |
|---|---|---|---|
| Number Lock | 4 digits | 0 to 9 | All different, plus clues (parity, bounds, order) |
| Object Arrangement | 4 objects | Positions 1 to 4 | All different, plus clues (forbidden, before, adjacent) |
| Logic Doors | 1 (exit) | A, B, C | Exactly one of three signs is true |

The clue pool for a puzzle is built from templates (Table IV). Every clue is created only after the secret solution is known, and only if it is true for that secret, so the secret always satisfies all clues. For the Number Lock, unary clues test parity or a bound on one digit, and binary clues compare two digits. For the arrangement, clues forbid a position, order two objects or state whether two objects are adjacent. For the doors, each sign says that the exit is, is not, or is one of two named doors.

**TABLE IV. CLUE TEMPLATES**

| Puzzle | Template (arity) | Example |
|---|---|---|
| Number Lock | Parity (1) | The second digit is even |
| | Bound (1) | The first digit is greater than 3 |
| | Order (2) | The first digit is bigger than the third |
| | Exact (1), fallback | The fourth digit is exactly 7 |
| Arrangement | Forbidden (1) | Key cannot be in position 2 |
| | Before (2) | Book must appear before Coin |
| | Adjacent / not (2) | Card must be next to Key |
| | Exact (1), fallback | Coin must be in position 4 |
| Doors | Is / is not / either (1) | The exit is NOT behind Door B |

### D. Algorithms

The engine has four functions. `checkConstraints()` tests a (possibly partial) assignment: a rule is evaluated only when all of its variables already have a value, and rules with unassigned variables are skipped. `backtrack()` chooses

XXX-X-XXXX-XXXX-X/XX/$XX.00 ©20XX IEEE

the first unassigned variable, tries each value in its domain, calls the checker, recurses if no rule is broken, and otherwise counts a backtrack and tries the next value. `solveCSP()` runs the search and returns up to a given number of solutions. `buildClues()` is the generator. Algorithms 1 and 2 give the pseudocode and Fig. 2 the flow.

```
function BACKTRACK(assignment):
  v <- first unassigned variable
  if v is none: save solution; return
  for value in domain[v]:
    assignment[v] <- value
    if CHECK CONSTRAINTS(assignment):
      BACKTRACK(assignment)
    else:
      backtracks <- backtracks + 1
    delete assignment[v]      # undo
    if solutions >= limit: return
```

**Algorithm 1. Backtracking search**

```
function BUILD CLUES(vars, domains, base, level):
  secret <- SOLVE(base, limit = 1)  # shuffled
  pool   <- shuffled clues TRUE for secret,
            then 'exact' clues as safety net
  cons   <- base
  while pool not empty and
        |SOLVE(cons, limit = 2)| > 1:
    cons <- cons + next clue from pool
  cons <- cons + EXTRA[level] more clues
  return cons, SOLVE(cons, limit = 1)
```

**Algorithm 2. Generate-and-verify clue builder**

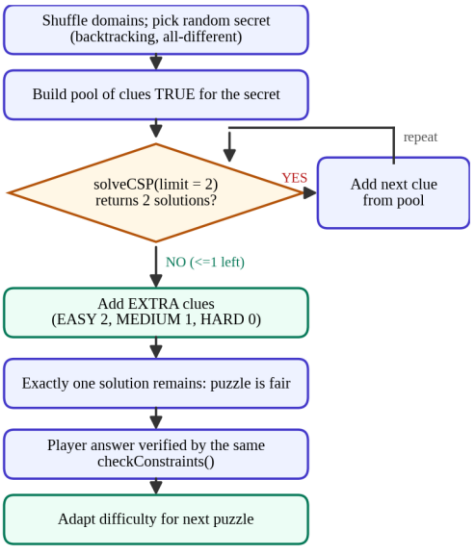



**Fig. 2. Flowchart of puzzle generation**

**Why the result is fair.** The secret satisfies every clue by construction, so at least one solution exists. The loop stops only when the solver can no longer find a second solution, so at most one exists. Hence exactly one solution remains. The exact-value clues at the end of the pool guarantee termination, because after all of them are added only the secret is left. Asking for two solutions instead of all solutions lets the solver stop as soon as the second one is found. For the doors, the generator draws random signs and accepts them when exactly one exit makes exactly one sign true, with up to 500 attempts.

### *E. Adaptive Difficulty*

For each puzzle the game records the number of wrong attempts, the solving time and the hints used. When the puzzle is solved, the rules in Table V update the level, which is EASY, MEDIUM or HARD (Fig. 3). The game starts at MEDIUM. The level sets how many extra clues are added after uniqueness is reached: 2, 1 and 0. A higher level therefore means fewer clues, and the new level applies from the next generated puzzle. The door puzzle does not use the level.

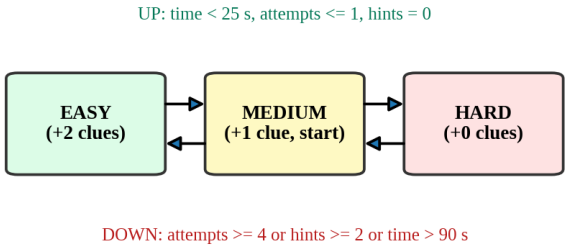



**Fig. 3. Difficulty levels and transition rules**

**TABLE V. ADAPTIVE DIFFICULTY RULES**

| Event | Condition | Effect |
|---|---|---|
| Fast, clean solve | Time < 25 s, attempts ≤ 1, hints = 0 | Level + 1 |
| Struggled solve | Attempts ≥ 4, or hints ≥ 2, or time > 90 s | Level − 1 |
| Otherwise | None of the above | Level kept |
| Stuck during a puzzle | Attempts ≥ 3 or time > 60 s | +1 extra hint (once) |

The struggle check runs every second and after every wrong answer. Each puzzle allows two hints by default, and each hint reveals one value of the secret solution (or, for the doors, rules out one wrong door). The extra hint raises this to three.

### *F. Transparency of the AI*

The AI Status panel shows two live counters, the number of backtracks and the number of constraint checks for the current puzzle, and a log of the last five AI events such as “Generated number lock with CSP + backtracking” or “Player struggled, difficulty down to EASY”. An AI Explanation screen describes CSP, backtracking, puzzle generation, constraint checking and adaptive difficulty, and ends with the loop *variables, possible values, constraint check, valid or backtrack*.

## IV. DETAILED METHODOLOGY

### *A. Number Lock*

The generator picks a secret code with backtracking over shuffled domains, builds the clue pool and calls `buildClues()`. The interface shows the clues, a display and a keypad. When the player presses OK, the four digits are written into an assignment and passed to `checkConstraints()` with the puzzle’s constraint list. No stored code is compared. A wrong code increases the attempt count and clears the entry; a correct code adds the lock code to the inventory.

### *B. Object Arrangement*

The player selects an object and then a slot, or clicks a filled slot to empty it. The check button builds an assignment from the slots and validates it in the same way. The position of the Key is stored for the final code.

### *C. Logic Doors*

Each door carries a sign. The rule is that exactly one sign tells the truth. The solver evaluates all three possible exits and keeps the puzzle only if one exit satisfies the rule. A wrong door gives a message that tells the player to count how many signs would be true.

### *D. Final Escape and User Interface*

The final stage concatenates the saved lock code, door letter and Key position and compares the result with the typed string. The interface has a timer, difficulty chip, a progress

XXX-X-XXXX-XXXX-X/XX/$XX.00 ©20XX IEEE

bar for the four stages, an inventory, a hint button that shows the hints remaining, and a victory screen with time, hints used and final difficulty. Keyboard focus outlines are visible, motion is disabled when the user prefers reduced motion, and the layout collapses to one column below 800 px.

## V. RESULTS AND DISCUSSION

### *A. Experimental Setup*

The generation experiments ran the unchanged engine and generator code under Node.js, which uses the same V8 JavaScript engine as Chrome. For each puzzle type and difficulty level, 2,000 puzzles were generated (12,000 in total), and 2,000 door puzzles were generated, giving 14,000 puzzles. Timing covers the whole generation, including every solver call. Uniqueness was checked by a separate brute-force routine that enumerates every candidate assignment and tests it against every clue directly, without using the backtracking solver.

**TABLE VI. EXPERIMENTAL SETUP**

| Parameter | Value |
|---|---|
| Runtime | Node.js v22.22.2 (V8) |
| Machine | 1 vCPU Intel Xeon 2.8 GHz, 3 GB RAM |
| Puzzles per configuration | 2,000 |
| Configurations | 2 puzzle types × 3 levels, plus doors |
| Search space | 5,040 (number lock); 24 (arrangement); 3 (doors) |
| Uniqueness check | Independent brute-force enumeration |

### *B. Generation Results*

Table VII gives the results. Every one of the 12,000 number and arrangement puzzles had exactly one solution under the brute-force check, and the secret was always that solution. Mean generation time was about 1 ms for the arrangement and 2.1 to 2.6 ms for the number lock, far below the delay a player can notice. The worst single generation took 34 ms. The door generator succeeded in all 2,000 runs with a mean of 1.93 attempts (maximum 13) and a mean time of 0.054 ms.

**TABLE VII. GENERATION RESULTS (N = 2,000 EACH)**

| Puzzle / level | Mean ms | p95 ms | Back-tracks | Checks | Clues | Unique |
|---|---|---|---|---|---|---|
| Number / EASY | 2.56 | 5.42 | 216 | 2,607 | 20.4 | 100% |
| Number / MEDIUM | 2.13 | 4.11 | 206 | 2,542 | 20.0 | 100% |
| Number / HARD | 2.19 | 4.24 | 207 | 2,546 | 19.4 | 100% |
| Arrange / EASY | 0.98 | 1.76 | 148 | 924 | 8.6 | 100% |
| Arrange / MEDIUM | 1.03 | 1.99 | 149 | 928 | 7.7 | 100% |
| Arrange / HARD | 1.06 | 2.02 | 147 | 916 | 6.7 | 100% |

**Back-tracks and checks are means per generation. Clues exclude the base all-different rule.**

### *C. Effect of Difficulty*

In the arrangement puzzle the mean number of clues falls from 8.6 at EASY to 6.7 at HARD, a drop of about 22% (Fig. 4). The number lock shows a much smaller drop, from 20.4 to 19.4, about 5%. The reason is that most clues come from the uniqueness loop, which adds clues until one solution is left. Only the extra clues depend on the level. The number lock also leans on its safety net: on average 3.0 of the 4 digits are given by exact-value clues at MEDIUM (3.3 at EASY and 2.5 at HARD), whereas the arrangement puzzle never needed them.

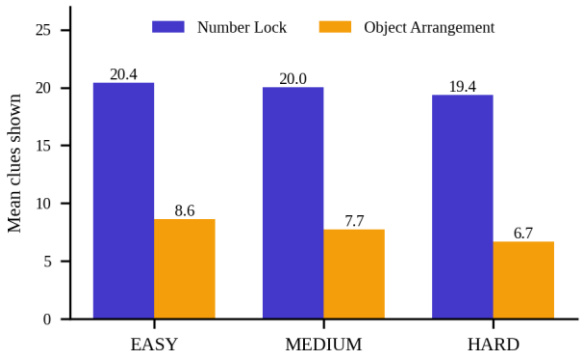
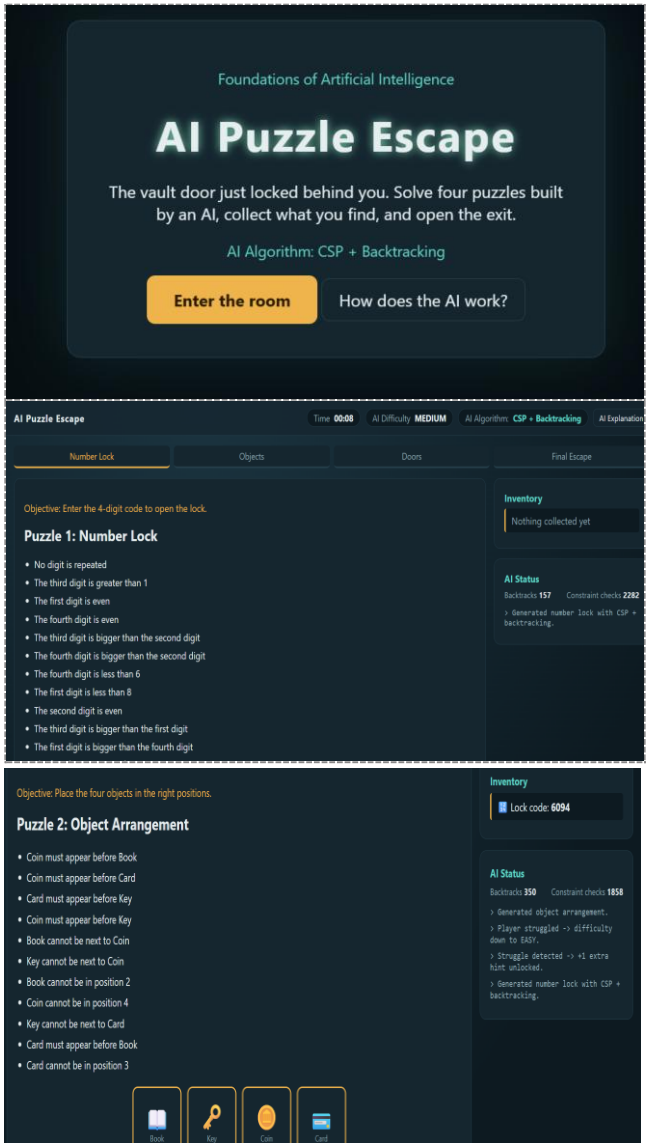



**Fig. 4. Mean number of clues shown at each level**

Applying the rules of Table V to these means gives the expected behaviour. A player who solves the Number Lock in 18 s with one attempt and no hints moves to HARD and receives an arrangement puzzle with about 6.7 clues. A player who needs 100 s moves to EASY and receives about 8.6 clues. A player with three wrong answers receives an extra hint at once. These are traces of the rules, not a user study.

### *D. Game Interface*



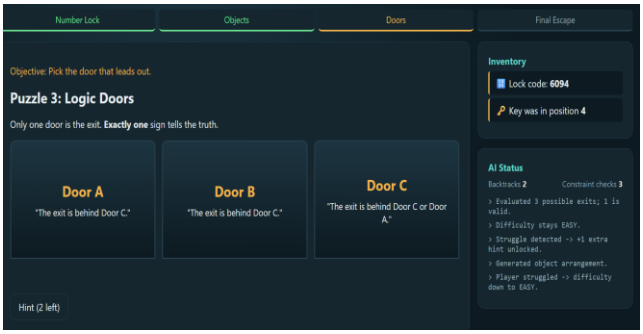



**Fig. 5. Game screen with clues, inventory and AI Status panel**

### *E. Discussion and Limitations*

The engine met its main goal: a fair puzzle on every run, with no stored answers and the same checker for generation and play. The results also show limits that should be reported honestly.

- **No human evaluation.** The adaptive rules were not tested with players, and the thresholds (25 s, 90 s, 4 attempts) were set by hand.
- **Basic search.** The solver uses a fixed variable order and no MRV or forward checking. With 4 variables this is instant, but it would not scale to larger puzzles.
- **Redundant clues.** The uniqueness loop does not remove clues that later become unnecessary, so the number lock shows about 20 clues and often reveals digits directly.
- **Narrow difficulty control.** Difficulty only changes the count of extra clues, and not at all for the doors.
- **Client-side answers.** The puzzle data is in browser memory, so a determined player could read it. This is acceptable for a learning game.
- **Timing source.** Times were measured in Node.js on one machine, not in each browser.

## VI. CONCLUSION AND FUTURE EXTENSIONS

This work built a complete escape game around a classical AI method. Puzzles are modelled as CSPs, created by choosing a secret, deriving true clues and adding them until backtracking finds one solution, and answers are validated by the same constraint checker. In 14,000 generated puzzles the method produced a uniquely solvable puzzle every time, in a few milliseconds, and an adaptive module varied the clue count with player performance. Showing the backtracks and constraint checks in the interface helps the player see how the AI works.

Future work could:

- Add MRV ordering and forward checking and test larger puzzles.
- Rank clues by how many candidates they remove and delete redundant ones, so that hard puzzles are truly harder.
- Add more puzzle families, such as cryptarithms or graph colouring.
- Run a user study and tune the difficulty thresholds from the data.
- Let difficulty change the search depth or the clue type, and apply it to the doors.
- Benchmark in several browsers and add a leaderboard.

## VII. REFERENCES

[1] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed. Hoboken, NJ, USA: Pearson, 2020.

[2] A. K. Mackworth, "Consistency in networks of relations," *Artificial Intelligence*, vol. 8, no. 1, pp. 99–118, 1977.

[3] R. M. Haralick and G. L. Elliott, "Increasing tree search efficiency for constraint satisfaction problems," *Artificial Intelligence*, vol. 14, no. 3, pp. 263–313, 1980.

[4] R. Dechter, *Constraint Processing*. San Francisco, CA, USA: Morgan Kaufmann, 2003.

[5] E. Tsang, *Foundations of Constraint Satisfaction*. London, U.K.: Academic Press, 1993.

[6] F. Rossi, P. van Beek and T. Walsh, Eds., *Handbook of Constraint Programming*. Amsterdam, The Netherlands: Elsevier, 2006.

[7] J. Togelius, G. N. Yannakakis, K. O. Stanley and C. Browne, "Search-based procedural content generation: A taxonomy and survey," *IEEE Trans. Comput. Intell. AI Games*, vol. 3, no. 3, pp. 172–186, 2011.

[8] A. M. Smith and M. Mateas, "Answer set programming for procedural content generation: A design space approach," *IEEE Trans. Comput. Intell. AI Games*, vol. 3, no. 3, pp. 187–200, 2011.

[9] T. Yato and T. Seta, "Complexity and completeness of finding another solution and its application to puzzles," *IEICE Trans. Fundamentals*, vol. E86-A, no. 5, pp. 1052–1060, 2003.

[10] R. Hunicke and V. Chapman, "AI for dynamic difficulty adjustment in games," in *Proc. AAAI Workshop on Challenges in Game AI*, 2004.

[11] M. Csikszentmihalyi, *Flow: The Psychology of Optimal Experience*. New York, NY, USA: Harper & Row, 1990.

[12] P. Sweetser and P. Wyeth, "GameFlow: A model for evaluating player enjoyment in games," *ACM Comput. Entertain.*, vol. 3, no. 3, art. 3, 2005.

XXX-X-XXXX-XXXX-X/XX/$XX.00 ©20XX IEEE